

0731 Answer | 增长量级九道题完整讲解

原题入口：[0731 题单](#) | [九题考场路线与保底](#) | 教学上下文：[0731-context.md](#) | 质量门槛：[quality-gate.md](#)

这不是九个答案的速记表。它保存的是从不会下手到能够判断的中间过程：第一眼看什么、怎样迈出第一步、哪些错误路线容易成立，以及最后如何收束。

01 | 2011-01：固定倍增为什么是对数

1. 设 n 是描述问题规模的非负整数，下面程序片段的时间复杂度是_____。

```
x=2;
while(x<n/2)
    x=2*x;
```

- A. $O(\log_2 n)$ B. $O(n)$ C. $O(n \log_2 n)$ D. $O(n^2)$

代码中真正反复发生的动作是 $x=2*x$ 。先不要急着认 while 或背结论，只写出 x 的变化：

2 → 4 → 8 → 16 →

每执行一轮， x 都多乘一个 2。执行 t 轮以后， x 大约是 $2^{(t+1)}$ ；循环在它达到 $n/2$ 量级时停止。因此问题变成：2 连乘多少次能够到达 n ？答案是约 $\log_2 n$ 次，所以选 A， $O(\log n)$ 。

排除时要看选项背后的代码条件。若是 $x++$ 或 $x+=\text{常数}$ ，才需要线性次数；本题不断翻倍，越往后走得越快。 $n/2$ 也不会把复杂度变成 $O(n)$ ，因为它只把终点缩小一个常数倍。

这题留下的动作是：看到固定倍增或减半，写出前几项，再问多少轮到达边界。

02 | 2012-01：递归不等于指数增长

1. 求整数 n ($n \geq 0$) 阶乘的算法如下，其时间复杂度是_____。

```
int fact(int n){
    if (n<=1) return 1;
    return n*fact(n-1);
}
```

- A. $O(\log_2 n)$ B. $O(n)$ C. $O(n \log_2 n)$ D. $O(n^2)$

阶乘函数返回的结果是 $n!$ ，但题目问的不是结果有多大，而是程序做了多少工作。把调用过程展开：

```
fact(n)
→ fact(n-1)
→ fact(n-2)
→ .....
→ fact(1)
```

每一层只继续调用一次，没有分叉。从 n 每次减 1 到达 1，大约经历 n 层；每层只有一次判断、一次乘法等常数工作，所以选 B， $O(n)$ 。

$O(\log n)$ 要求参数类似 $n/2$ ，这样才能不断减半； $O(n \log n)$ 通常需要对数层中每层再完成线性工作； $O(n^2)$ 则需要每层还有一个随 n 增长的循环。本题都没有。

这题留下的动作是：看到递归，先画调用链或调用树；数调用次数，不看返回值有多大。

03 | 2013-01：没有代码时，先想数据会被碰几次

1. 已知两个长度分别为 m 和 n 的升序链表，若将它们合并为一个长度为 $m + n$ 的降序链表，则最坏情况下的时间复杂度是_____。

- A. $O(n)$ B. $O(mn)$ C. $O(\min(m, n))$ D. $O(\max(m, n))$

题干要按顺序拆开。“两个升序链表”说明两边都已经排好，不必重新排序；合并时只需比较当前结点并让指针向后推进。“合并成降序”改变输出的组织方向，但不必因此反复扫描输入。“最坏情况”则要求考虑两条链表的结点都需要进入结果。

长度分别为 m 和 n ，每个结点至多被处理一次，所以总工作量是：

$$m + n$$

选项给的是 $O(\max(m, n))$ 。因为：

$$\max(m, n) \leq m + n \leq 2 \times \max(m, n)$$

二者只差一个不超过 2 的常数倍，在渐进复杂度中同阶，所以选 D。

$O(mn)$ 容易来自“两个链表就是两层循环”的外观判断，但有序合并的两个指针都只向前，不会让一边的每个结点重新遍历另一边。 $O(\min(m, n))$ 又漏掉了较长链表剩余部分仍需接入结果。

这题留下的动作是：没有现成代码时，先构造一个合理过程，再数每个数据对象会不会被重复访问。

04 | 2014-01：什么时候两层循环可以相乘

1. 下列程序段的时间复杂度是_____。

```
count=0;
for (k=1; k<=n; k*=2)
    for (j=1; j<=n; j++)
        count++;
```

- A. $O(\log_2 n)$ B. $O(n)$ C. $O(n \log_2 n)$ D. $O(n^2)$

先确认题目在算谁：算的是整段程序随输入规模 n 增长的运行时间。 $\text{count}++$ 是循环体中的基本动作，它执行多少次，就代表这段循环做了多少份工作；不是研究 count 这个变量增长得快不快。

外层变量每轮翻倍，因此有 $O(\log n)$ 轮。内层的上界始终是 n ，不随外层变化，所以外层的每一轮都固定执行 n 次：

$$n + n + \dots + n (\text{共 } \log n \text{ 项}) = n \log n$$

因此选 C， $O(n \log n)$ 。这里能够直接相乘，并不是因为“看见两层循环”，而是因为内层每一轮的工作量确实相同。

还要分清变量的值与运行时间。即使把循环体改成 $\text{count}+=j$ ， count 的最终数值会改变，但循环执行次数没有改变，时间复杂度仍相同。

这题留下的动作是：先数外层轮数，再检查内层是否每轮固定；固定才能直接相乘。

05 | 2017-01：累计到 n ，需要反推进行了多少轮

1. 下列函数的时间复杂度是_____。

```
int func(int n){
    int i=0, sum=0;
    while(sum < n) sum += ++i;
    return i;
}
```

A. $O(\log n)$

B. $O(n^{1/2})$

C. $O(n)$

D. $O(n \log n)$

这类题没有思路时，先执行前三轮，不需要一上来记公式。 $++i$ 是先把 i 加一，再把它加入 sum ：

第1轮： $i=1, sum=1$

第2轮： $i=2, sum=1+2$

第3轮： $i=3, sum=1+2+3$

执行 t 轮后：

$$sum = 1 + 2 + \dots + t = t(t + 1) / 2$$

它的主要增长部分是 t^2 。循环在 sum 达到 n 时停止，因此反过来解：

$$t^2 \approx n \rightarrow t \approx \sqrt{n}$$

程序运行的是 t 轮，所以选 B， $O(\sqrt{n})$ 。这里不能看到 $i++$ 就选 $O(n)$ ，因为 i 并不需要走到 n ；不断累加的 sum 会先达到 n 。

这题留下的动作是：停止条件检查累计量时，先求执行 t 轮后的累计结果，再由终点反解 t 。

06 | 2019-01：条件中的平方是在限制轮数

1. 设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是_____。

```
x=0;
while (n >= (x+1) * (x+1))
    x=x+1;
```

A. $O(\log n)$

B. $O(n^{1/2})$

C. $O(n)$

D. $O(n^2)$

x 每轮只增加 1，但它不需要增加到 n 。循环条件要求：

$$(x + 1)^2 \leq n$$

也就是：

$$x + 1 \leq \sqrt{n}$$

因此 x 从 0 逐一走到约 $\sqrt{n}$ ，一共执行 $O(\sqrt{n})$ 轮，所以选 B。

这道题有两个方向相反的坑。一个是看到 $x++$ 就选 $O(n)$ ，忘了先求它实际走到哪里；另一个是看到平方就选 $O(n^2)$ ，把停止条件中的平方误当成工作量的平方。平方在这里让循环更早停止。

这题留下的动作是：更新方式决定怎样走，停止条件决定走到哪里；两者必须一起看。

07 | 2022-01：外层是 $\log n$ 轮，为什么总量仍是 n

01. 下列程序段的时间复杂度是（ ）。

```
int sum = 0;
for (int i = 1; i < n; i *= 2)
    for (int j = 0; j < i; j++)
        sum++;
```

- A. $O(\log n)$ B. $O(n)$ C. $O(n \log n)$ D. $O(n^2)$

外层 $i*=2$ ，所以 i 的实际取值是：

1, 2, 4, 8,

外层确实只有 $O(\log n)$ 轮。但内层并不是固定执行 n 次，而是执行当前的 i 次。因此各轮真正的工作量为：

$1 + 2 + 4 + 8 + \dots + L = 2L - 1$

其中 L 是小于 n 的最大 2 的幂，满足 $n/2 \leq L < n$ 。只有 n 恰为 2 的幂时，才有 $L=n/2$ 、总和 $n-1$ ；一般情况仍是 $\Theta(n)$ ，所以选 B， $O(n)$ 。

最关键的区别是：程序变量 i 不是“轮数”。第 k 轮时， i 约为 $2^{(k-1)}$ ，所以第 k 轮做的是 $2^{(k-1)}$ 次，并不是 k 次。也不能写成 $i \times \log n$ ，因为 i 每一轮都在变化，没有一个固定的 i 可以拿出来相乘。

如果真的是一共 $\log n$ 轮，并且每轮固定做 $\log n$ 次，结果才是 $O(\log^2 n)$ 。本题不是这种结构。

这题留下的动作是：内层上界依赖外层变量时，把外层变量的实际取值写出来并求和，不用最大值冒充每一轮。

08 | 2023-01：直接定位和完成整个操作不是一回事

01. 下列对顺序存储的有序表（长度为 n ）实现给定操作的算法中，平均时间复杂度为 $O(1)$ 的是（ ）。

- A. 查找包含指定值元素的算法 B. 插入包含指定值元素的算法
C. 删除第 i ($1 \leq i \leq n$) 个元素的算法 D. 获取第 i ($1 \leq i \leq n$) 个元素的算法

题目中的两个词要分开。“顺序存储”表示元素在内存中连续排列；“有序表”表示元素按关键字有序。决定本题 $O(1)$ 操作的是前者，不是后者。

顺序存储中，已知序号 i ，可以直接计算第 i 个元素的地址：

第 i 个元素地址 = 首地址 + $(i - 1) \times$ 每个元素所占空间

因此取得第 i 个元素不需要从头寻找，选 D，平均时间复杂度为 $O(1)$ 。

按值查找即便利用有序性进行折半，也通常是 $O(\log n)$ ；插入指定值需要先确定位置并移动后续元素；删除第 i 个元素虽然能立刻定位，但删除后还要搬移后面的元素。因此“能在 $O(1)$ 时间找到位置”不等于“整个操作是 $O(1)$ ”。

这题留下的动作是：把一个操作拆成定位成本和后续处理成本，二者相加才是总复杂度。

09 | 2025-01：根号 n 轮，每轮做 i 次，为什么是 n

1. 下列程序段的时间复杂度是

```
int count = 0; i, j;
for( i=1; i*i<= n; i++)
    for( j=1; j<=i; j++)
        count++;
```

A. $O(\log n)$

B. $O(n)$

C. $O(n \log n)$

D. $O(n^2)$

先看外层条件 $i*i \leq n$ 。它意味着 i 最大走到约 $\sqrt{n}$ ，所以外层有 $O(\sqrt{n})$ 轮。再看内层：第 i 轮执行 i 次，因此不能把两层都笼统写成某个固定复杂度，而要相加：

$$\begin{aligned} &1 + 2 + 3 + \dots + \sqrt{n} \\ &= \sqrt{n}(\sqrt{n} + 1) / 2 \\ &= O(n) \end{aligned}$$

所以选 B， $O(n)$ 。这道题把第六题的“先从平方条件解出边界”和第七题的“内层变化时逐轮求和”合到了一起。

从代码设计角度看，它也能反过来理解：如果第 i 轮做 i 次，外层走到 t 时总量约为 t^2 ；若希望总量约为 n ，便令 $t \approx \sqrt{n}$ ，代码中可以用 $i*i \leq n$ 表达这个边界。这不是说真实算法通常为了凑复杂度而写，而是帮助建立“边界、更新方式、每轮工作量”如何共同决定总工作量的代码感觉。

这题留下的动作是：先解外层最大值，再把它代入内层总和；不要只数循环层数。

对第 07、09 题最后一次澄清

两题都是 $O(n)$ ，但过程并不相同：

结构	每轮工作量	总和
外层翻倍，共 $\log n$ 轮	$1, 2, 4, \dots, L$ ，其中 $n/2 \leq L < n$	$O(n)$
外层递增到 $\sqrt{n}$	$1, 2, 3, \dots, \sqrt{n}$	$O(n)$
假设有 $\log n$ 轮，每轮固定做 $\log n$ 次	$\log n$ 重复 $\log n$ 轮	$O(\log^2 n)$

$\Theta(\log^2 n)$ 的增长确实严格慢于 n ，所以数学上有 $\log^2 n = o(n)$ ，也有 $\log^2 n = O(n)$ 。但复杂度选择题默认寻找最紧的量级；若程序实际是 $\Theta(\log^2 n)$ ，就不能用宽泛的 $O(n)$ 代替准确答案。

本组最终留下的考场动作

没有思路时，不猜选项，只在草稿上补完三行：

控制变量实际经过：_____

每一轮执行次数：_____

全部轮次相加：_____

若题目没有代码，就改为追问：每个数据对象会被访问、比较或移动几次。0731 到这里结束；后续题目继承的是这个思考动作，不是机械套用这九道题的答案。